AUTOMATED REVIEW RATING SYSTEM

1.Introduction

The Automated Review Rating System is an intelligent web application designed to predict a product's rating based on user-submitted text reviews. Using a fine-tuned BERT deep learning model, the system analyzes the sentiment and automatically generates an accurate rating score. The backend is built with Django, which handles model inference, API creation, and secure data storage. The predicted ratings and reviews are stored in a database for further analytics and monitoring. The frontend, developed using React, provides a clean and interactive user interface for entering reviews and viewing predictions. This system streamlines the review-rating process and enhances user experience with real-time, AI-powered insights**.**

2.Data Collection and Preparation

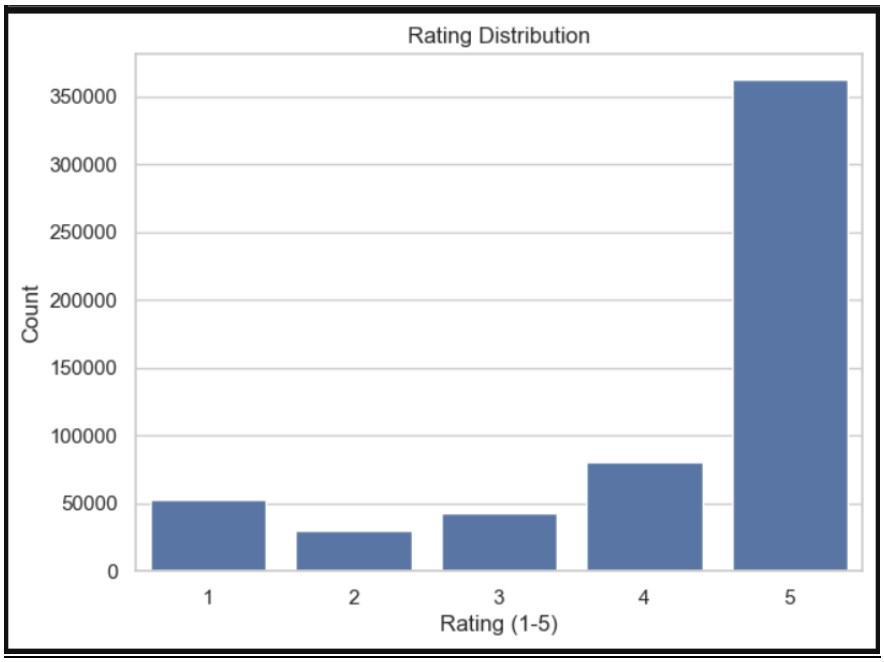
For this project, the Amazon Fine Food Reviews Dataset was used, containing **560,354** reviews and **10** structured columns, making it suitable for large-scale sentiment and rating prediction. The dataset was downloaded from Kaggle and validated to ensure completeness and consistency**. Missing values** and **duplicate entries** were identified and **removed to maintain data integrity**. Irrelevant or corrupted records were filtered out to avoid noise during model training. The cleaned dataset was then **split into training** and **testing sets using an 80-20 ratio for balanced evaluation**. All preprocessing steps—including text normalization, handling null values, and dataset verification—were documented for reproducibility.

Code of missing values and Text Preprocessing

```
df['ProfileName'] = df['ProfileName'].fillna(df['ProfileName'].mode()[0])

df['Summary'] = df['Summary'].fillna(df['Summary'].mode()[0])

from tqdm import tqdm

import re

from bs4 import BeautifulSoup

preprocessed_review = []


for sentence in tqdm(final_dataset['Text'].values):

  sentence = re.sub(r"http\s+","",sentence)

  sentence = BeautifulSoup(sentence, 'lxml').get_text()

  sentence = re.sub('[^a-zA-Z]+',' ', sentence)

  sentence = ' '.join(e.lower() for e in sentence.split() if e.lower() not in stopwords)

  preprocessed_review.append(sentence.strip())
```

3.EDA

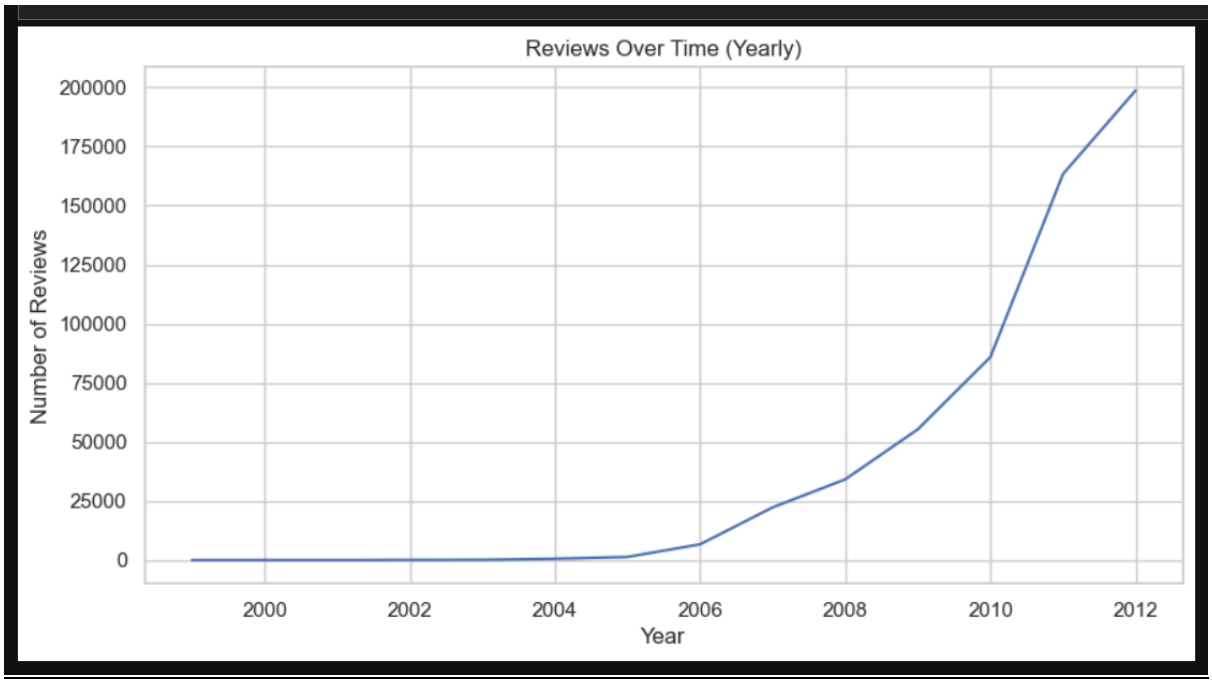
Rating Distribution



Average review length by rating

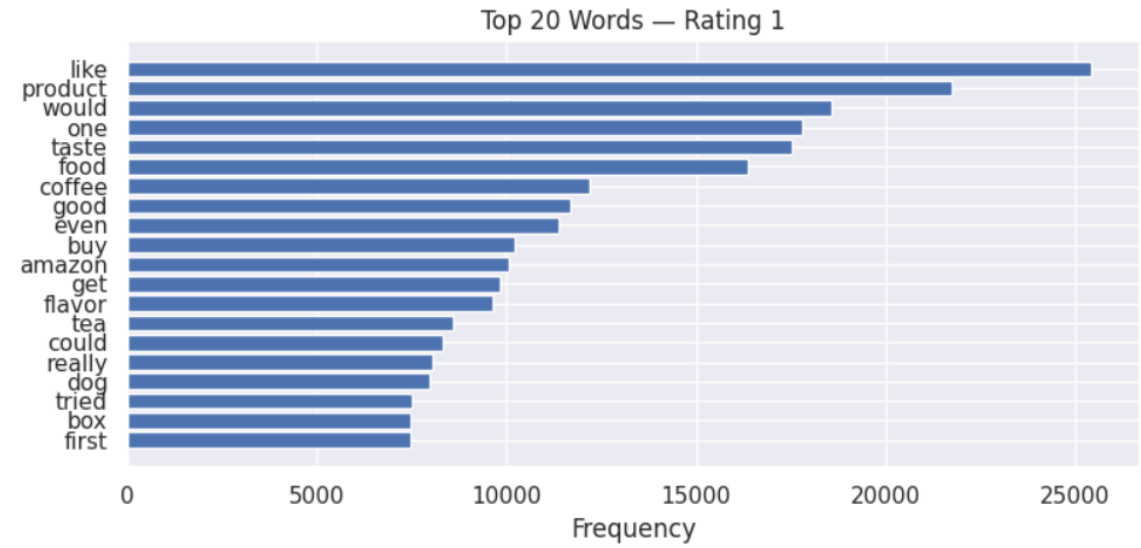
```
Average Review Length by Rating:
 Score
1    478.082364
2    490.655010
3    520.319841
4    496.603273
5    402.447373
Name: review_length, dtype: float64
```

Temporal trends (reviews over time)



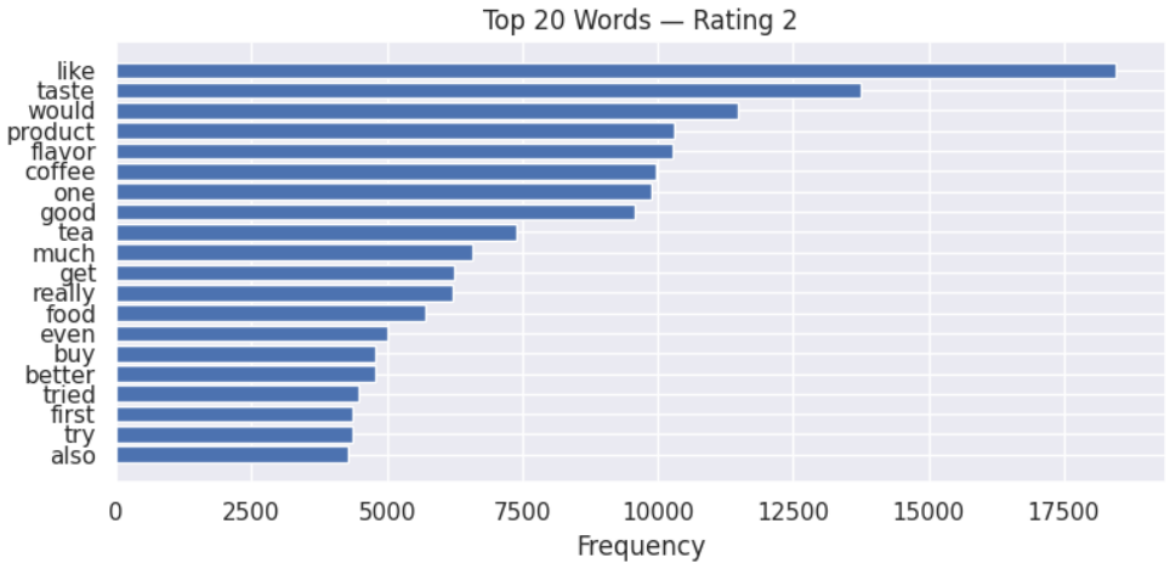
Top 20 words for rating 1:

like: 25417, product: 21750, would: 18561, one: 17808, taste: 17506, food: 16364, coffee: 12203, good: 11701, even: 11388, buy: 10215, amazon: 10059, get: 9811, flavor: 9644, tea: 8601 could: 8323, really: 8060, dog: 7956, tried: 749, box: 7460, first: 7459



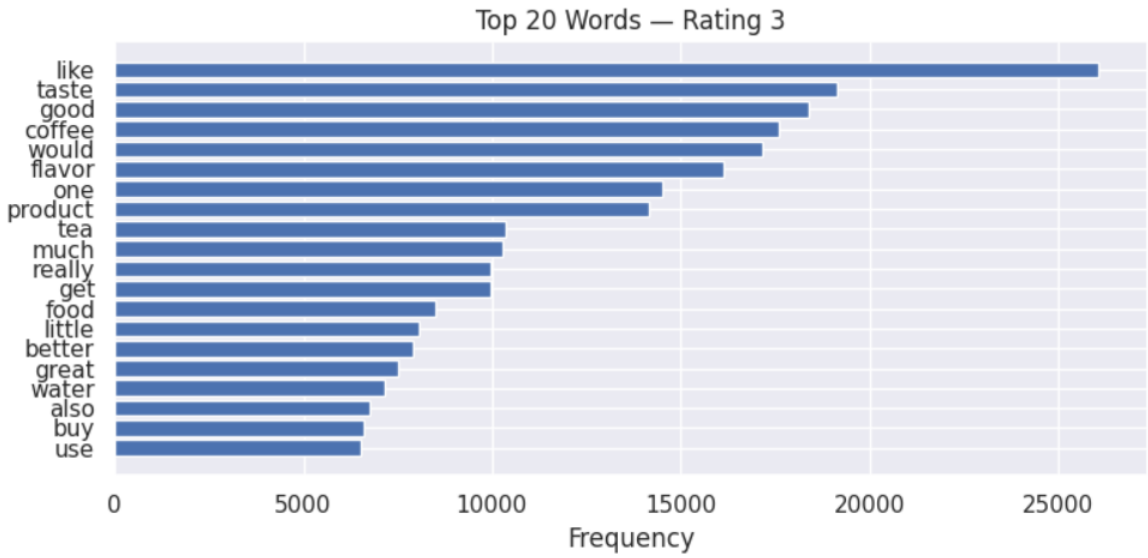
Top 20 words for rating 2:

like: 18459, taste: 13751, would: 11493, product: 10299, flavor: 10266, coffee: 9962, one: 9879, good: 9580, tea: 7388, much: 6575, get: 6252, really: 6218, food: 5716, even: 5020, buy: 4777 better: 4774, tried: 4484, first: 4360, try: 4360, also: 4272



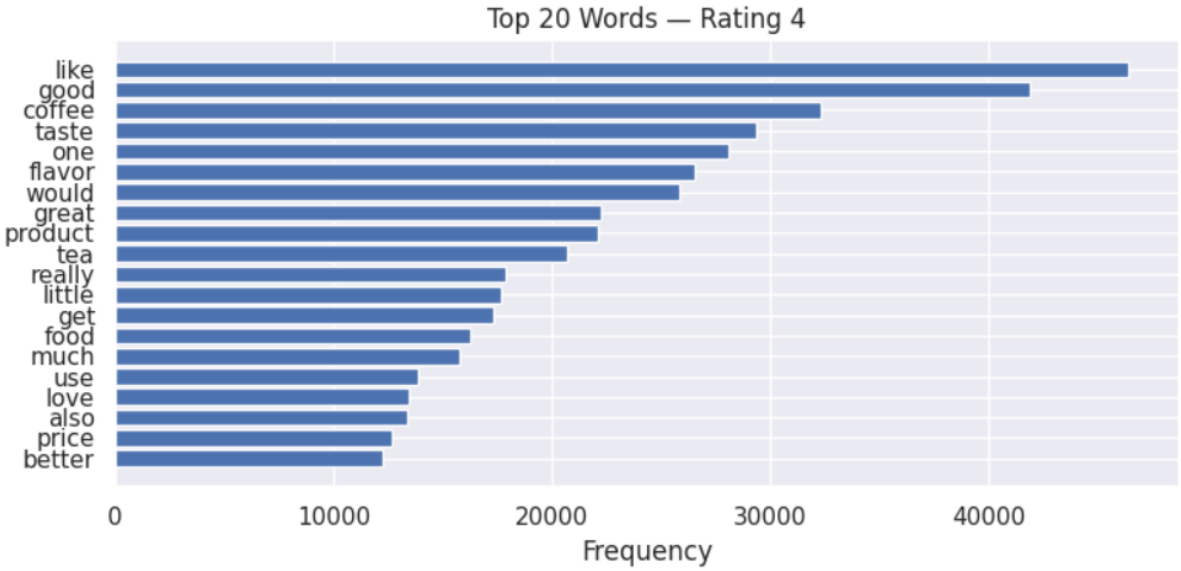
Top 20 words for rating 3:

like: 26076, taste: 19155, good: 18393, coffee: 17616, would: 17168,flavor: 16143, one: 14498, product: 14151, tea: 10366, much: 10288, really: 9960, get: 9949, food: 8491, little: 8048, better: 7900, great: 7516, water: 7151, also: 6762, buy: 6607, use: 6527



Top 20 words for rating 4:

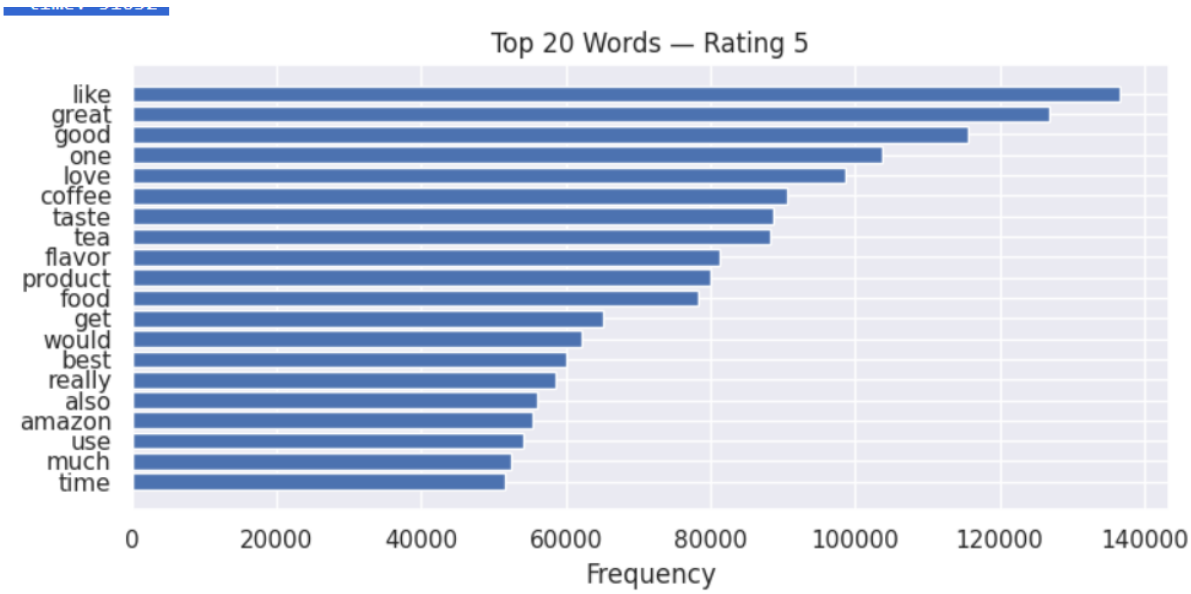
like: 46414, good: 41868, coffee: 32333, taste: 29328, one: 28081, flavor: 26544, would: 25834, great: 22262, product: 22093, tea: 20673, really: 17908, little: 17693, get: 17309, food: 16241, much: 15793, use: 13883, love: 13472, also: 13361, price: 12641, better: 12216



Top 20 words for rating 5:

like: 136546, great: 126840, good: 115701, one: 103812, love: 98638,coffee: 90702, taste: 88606, tea: 88248, flavor: 81342, product: 80057,food: 78245, get: 65103, would: 62144, best: 60012, really: 58590,

also: 56114, amazon: 55445, use: 54131, much: 52535, time: 51632



Word_Cloud – 1 star Rating

Word_Cloud 5 star Rating

Train-Test-Split

Train–test split divides the dataset into two parts: a training set used to teach the model patterns, and a test set used to evaluate how well the model generalizes to unseen data. This prevents overfitting and gives a realistic estimate of real-world performance. A common split ratio is 80% for training and 20% for testing.

Code

```
X = tf_idf_vect_counts

y = final_dataset['Score']

from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
    shuffle=True
)
```

Vectorization Used In Models

1.Machine Learning

- TF-IDF
  TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors by measuring how important each word is in a document relative to the entire dataset. It increases the weight of unique, meaningful words while reducing the influence of common words like “the” or “very.” This helps the model focus on informative terms and improves classification performance.

  Code

  ```
  from sklearn.feature_extraction.text import TfidfVectorizer

  # Initialize TF-IDF vectorizer
  tf_idf_vect = TfidfVectorizer(max_features=50000, ngram_range=(1,2), min_df=20)
  tf_idf_vect.fit(preprocessed_review)
  print('Sample Feature Names:', tf_idf_vect.get_feature_names_out()[1500:1550])
  print("="*60)
  tf_idf_vect_counts = tf_idf_vect.transform(preprocessed_review)

  print("Type:", type(tf_idf_vect_counts))
  print("Shape:", tf_idf_vect_counts.shape)
  print("Number of Features:", tf_idf_vect_counts.shape[1])
  ```

2.Deep Learning

- Glove
- **GloVe (Global Vectors for Word Representation)** is a pre-trained word embedding model that captures semantic meaning based on global word co-occurrence statistics. Unlike TF-IDF, which is sparse, GloVe produces dense vector representations where similar words have similar embeddings. These embeddings help deep learning models like LSTM understand relationships between words more effectively. In NLP tasks, GloVe improves contextual understanding and enhances model performance compared to basic text vectorization methods.
- **Code**

```
glove_path = "/content/glove.6B.300d.txt"
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

max_words = 20000
max_len = 100

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post')

embedding_index = {}
with open("glove.6B.100d.txt", encoding="utf8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embedding_index[word] = coefs

embedding_dim = 100
word_index = tokenizer.word_index
num_words = min(max_words, len(word_index) + 1)

embedding_matrix = np.zeros((num_words, embedding_dim))
for word, i in word_index.items():
    if i >= max_words:
        continue
    embedding_vector = embedding_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
```

3.Transformer model

- **subword tokenization (WordPiece)**
  Transformers use subword tokenization (WordPiece) instead of TF-IDF. The text is broken into meaningful subword units and converted into integer token IDs. These token IDs represent the input to the DistilBERT model, which then generates contextual embeddings. This vectorization method preserves word meaning and handles unknown words effectively.

- Code

  ```
  tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
  ```

4.Model Development

In this project, multiple machine learning and deep learning models were developed to predict review ratings based on text. Traditional ML models such as **Logistic Regression** and **Support Vector Machine (SVM)** were first implemented as baselines, using TF-IDF features for review text. To capture sequential information in customer reviews, a deep learning-based **RNN-LSTM model** was trained using word embeddings. Further performance improvement was achieved with **Transformer-based DistilBert** which understands contextual meaning at a deeper level. Each model was trained, tuned, and evaluated to compare accuracy, precision, recall, and overall predictive capability. The final system highlights how modern NLP models significantly outperform traditional approaches for sentiment and rating prediction.

4.1 Used Models

1.Machine Learning

- Logistic Regression
- SVM

2.Deep Learning

- RNN (BiLSTM)
- Transformer Model (DistilBert)

4.2 step by step model explanation

- Logistic Regression
  Logistic Regression is a statistical classification model used as a baseline for text-based sentiment and rating prediction. It works by learning the relationship between input features—such as TF-IDF vectors of review text—and the probability of each rating class. Despite being simple and fast, it often performs surprisingly well on high-dimensional text data. In this project, Logistic Regression served as an initial benchmark to compare against more complex deep learning models.

- Results – Logistic Regression
  - Training Accuracy – 80%
  - Test Accuracy – 77%
  - Precision – 75%
  - Recall – 77%
  - F1-Score – 75%

- Code

```
from sklearn.linear_model import LogisticRegression

lg = LogisticRegression(
    multi_class='multinomial',
    solver='lbfgs',
    max_iter=1000,
    n_jobs=-1,

)
lg.fit(X_train, y_train)
-----------------------------------------------------------------------------
y_pred_train = lg.predict(X_train)
y_pred_test = lg.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Accuracy
print("Training Accuracy:", accuracy_score(y_train, y_pred_train))
print("************************")
print("Test Accuracy:", accuracy_score(y_test, y_pred_test))

# Classification Reports
print("\nTraining Classification Report:")
print(classification_report(y_train, y_pred_train))

print("\nTest Classification Report:")
print(classification_report(y_test, y_pred_test))
```

- SVM (Support Vector Machine)

- **Support Vector Machine (SVM)** is a powerful supervised learning algorithm widely used for text classification tasks. It works by finding the optimal hyperplane that separates different rating classes in a high-dimensional feature space. SVM performs particularly well with sparse text representations like TF-IDF, making it suitable for sentiment and review analysis. In this project, SVM provided a strong traditional machine-learning baseline before moving to deep learning models.

  Results – SVM
  - Training Accuracy – 88%
  - Test Accuracy – 71%
  - Precision – 76%
  - Recall – 71%
  - F1-Score – 73%
- Code

```
from sklearn.svm import LinearSVC

linear_svc = LinearSVC(
    max_iter=2000,
    class_weight='balanced',
    random_state=42
)
```

```
linear_svc.fit(X_res, y_res)

# Predict on resampled training set
y_pred_train_svm = linear_svc.predict(X_res)

# Predict on original test set
y_pred_test_svm = linear_svc.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Training Accuracy
print("Training Accuracy:", accuracy_score(y_res, y_pred_train_svm))

# Test Accuracy
print("Test Accuracy:", accuracy_score(y_test, y_pred_test_svm))

print("\nTraining Classification Report:")
print(classification_report(y_res, y_pred_train_svm))

print("\nTest Classification Report:")
print(classification_report(y_test, y_pred_test_svm))
```

- Results Screenshot

```
Training Accuracy: 0.8836081487652239
Test Accuracy: 0.7128268728395387

Training Classification Report:
              precision    recall  f1-score   support

           1       0.93      0.95      0.94    290498
           2       0.92      0.95      0.93    290498
           3       0.88      0.91      0.90    290498
           4       0.83      0.80      0.81    290498
           5       0.84      0.81      0.83    290498

    accuracy                           0.88   1452490
   macro avg       0.88      0.88      0.88   1452490
weighted avg       0.88      0.88      0.88   1452490


Test Classification Report:
              precision    recall  f1-score   support

           1       0.66      0.74      0.70     10454
           2       0.41      0.55      0.47      5954
           3       0.42      0.57      0.48      8528
           4       0.44      0.58      0.50     16131
           5       0.92      0.77      0.84     72624

    accuracy                           0.71    113691
   macro avg       0.57      0.64      0.60    113691
weighted avg       0.76      0.71      0.73    113691
```

```
Training Accuracy: 0.8183999138012548
***********************
Test Accuracy: 0.7810644642056099

Training Classification Report:
              precision    recall  f1-score   support

           1       0.78      0.80      0.79     41814
           2       0.77      0.46      0.57     23815
           3       0.71      0.51      0.59     34112
           4       0.72      0.44      0.55     64524
           5       0.84      0.97      0.90    290498

    accuracy                           0.82    454763
   macro avg       0.77      0.63      0.68    454763
weighted avg       0.81      0.82      0.80    454763


Test Classification Report:
              precision    recall  f1-score   support

           1       0.72      0.74      0.73     10454
           2       0.62      0.33      0.43      5954
           3       0.59      0.41      0.48      8528
           4       0.62      0.37      0.46     16131
           5       0.83      0.96      0.89     72624

    accuracy                           0.78    113691
   macro avg       0.68      0.56      0.60    113691
weighted avg       0.76      0.78      0.76    113691
```

4.2 Deep Learning Models

Deep Learning models learn complex patterns from review text by automatically extracting semantic and contextual features. Unlike traditional ML models, they do not rely solely on manually engineered features such as TF-IDF. Architectures like **RNN–LSTM** capture long-term dependencies in sentences, while **Transformer models such as BERT** understand context using attention mechanisms. In this project, deep learning significantly improved rating prediction accuracy by leveraging richer language understanding.

- LSTM

  Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network designed to capture long-term dependencies in text. It overcomes the limitations of traditional RNNs by using memory cells and gating mechanisms that retain important information over long sequences. This makes LSTM highly effective for understanding sentiment and patterns in customer reviews. In this project, the LSTM model helped learn contextual meaning from review text, improving rating prediction accuracy.

  Results – RNN(LSTM)
  - Train Accuracy – 82%
  - Test Accuracy – 78%
  - Precision – 78%
  - Recall – 78%
  - F1-Score – 76%
- Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout

num_classes = 5  # Ratings 1-5

model = Sequential()
model.add(Embedding(input_dim=num_words,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=max_len,
                    trainable=False))  # Freeze GloVe
model.add(Bidirectional(LSTM(128, return_sequences=True)))
model.add(Dropout(0.5))
model.add(Bidirectional(LSTM(64)))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.summary()
```

- Transformer Model

  The **Transformer** is a modern deep learning architecture that relies on self-attention mechanisms to understand relationships between words in a sentence. Unlike RNNs and LSTMs, Transformers process all words in parallel, making them faster and more effective for long-text understanding. Models like **BERT, GPT, and RoBERTa** are all based on the Transformer architecture. In this project, the Transformer-based BERT model provided strong contextual understanding and improved rating prediction accuracy.

  Results – Transformer(BERT)

  - Training Accuracy – 80%
  - Test Accuracy – 80%
  - Precision – 78%
  - Recall – 78%
  - F1-Score – 78%

Finalised Model

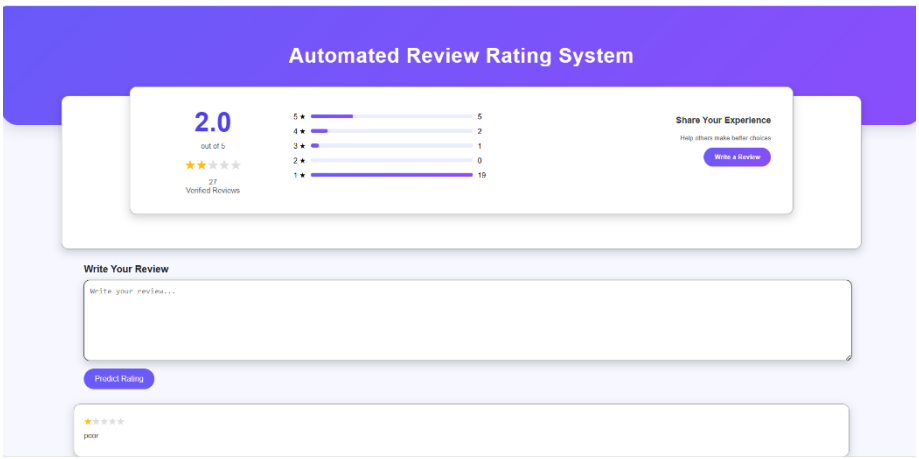
BERT (Bidirectional Encoder Representations from Transformers) was finalized as the best model because it understands text context in both forward and backward directions, giving it superior language comprehension. Compared to traditional ML models and LSTM, BERT captures deeper semantic meaning and subtle sentiment cues in reviews. It consistently delivered higher accuracy, precision, and robustness across all rating classes. Due to its strong contextual understanding and overall performance, BERT was selected as the final model for the Automated Review Rating System.

Development Part

- **Front end – React(Javascript)**

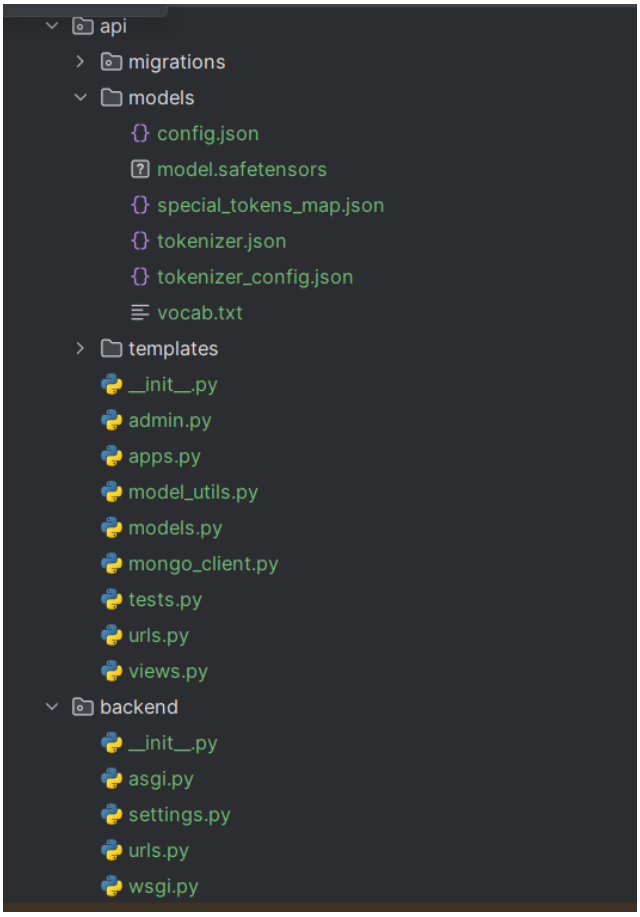
  The frontend of the project is developed using **React**, offering a smooth and responsive user experience for submitting and viewing product reviews. It includes components for the review form, review list, and predicted rating display. The interface communicates with the backend API to send user input and show the model's predictions in real time. Custom CSS is used to style the layout, making it clean, modern, and user-friendly. The design focuses on simplicity and accessibility so users can easily interact with the system.

- Sample



- **Backend-Django(Python)**
  The backend is built using **Django**, a powerful Python web framework that handles all API requests from the React frontend. It processes user-submitted reviews, runs the trained machine-learning model, and returns the predicted rating as a JSON response. Django REST Framework (DRF) is used to create clean and structured API endpoints. The backend ensures secure data handling, smooth communication with the model, and fast prediction delivery. Overall, Django provides a stable and scalable environment for connecting the ML model with the frontend.

  Sample



5. ANALYSIS & INSIGHTS

1.Which model performs best and why?

Among all the models evaluated, **BERT performed the best** for the review rating prediction task. Its Transformer-based architecture enables it to understand context bidirectionally, capturing subtle sentiment cues and complex language patterns that traditional models often miss. Unlike TF-IDF or LSTM-based approaches, BERT leverages deep contextual embeddings, resulting in significantly higher accuracy and balanced performance across all rating classes. Its ability to generalize well on unseen reviews made it the most reliable and robust choice for the final system.

2.Which rating is hardest to predict and why?

In this project, **mid-range ratings such as 3-star reviews were the hardest to predict**. These reviews often contain a mix of positive and negative expressions, making the sentiment ambiguous and less polarized. Unlike 1-star or 5-star reviews, which have strong emotional cues, 3-star reviews tend to be neutral, balanced, or inconsistent in tone. This lack of clear sentiment patterns reduces model confidence and increases misclassification.

3. Most important features contributing to predictions?

The most important features contributing to the model's predictions were **contextual word embeddings** that capture the meaning and sentiment of review text. Strong sentiment words such as *"excellent," "terrible," "love," "bad,"* and *"disappointed"* played a major role in identifying extreme ratings. For traditional ML models, TF-IDF features like frequently occurring adjectives and sentiment-heavy phrases were highly influential. In deep learning models like BERT, attention weights helped identify key phrases and contextual cues that strongly indicated the review's rating.

4. Error analysis: What reviews does the model misclassify?

The model most commonly misclassified **neutral or mixed-opinion reviews**, where the text contains both positive and negative statements. Reviews that are **short, vague, or lack clear sentiment cues** also caused confusion, making it difficult for the model to assign the correct rating. Mid-range ratings like **3-stars** were especially prone to errors because their sentiment overlaps with 2-star or 4-star reviews. Additionally, **sarcasm, idioms, and context-dependent expressions** sometimes misled the model, resulting in incorrect predictions.

5. Confusion between which rating pairs (1-2, 4-5)?

The model shows the most confusion between **adjacent rating pairs**, especially **1–2 stars** and **4–5 stars**, because their language patterns are often similar in tone. Reviews rated **1 or 2 stars** both express dissatisfaction, making it harder for the model to distinguish the severity of negativity. Similarly, **4 and 5-star** reviews share highly positive language, causing overlap in predicted sentiment strength. This leads to misclassifications where the model predicts a nearby rating rather than the exact one.

6. Business insights: What drives 5-star vs 1-star ratings?

5-star ratings are typically driven by clear positive themes such as product quality, fast delivery, good customer service, and overall satisfaction with performance or value. Customers often highlight reliability, ease of use, and exceeding expectations.
1-star ratings, on the other hand, are strongly linked to issues like poor product quality, defects, delays in delivery, misleading descriptions, or bad customer support. Negative experiences, unmet expectations, and frustration with service processes commonly trigger the lowest ratings.